

The Labyrinth

Technical Design Document (TDD)

****Versão:**** 1.0

****Engine:**** Godot 4.x

****Linguagem:**** GDScript

****Status:**** Em desenvolvimento

Índice

1. Objetivo
2. Stack do Projeto
3. Estrutura de Pastas
4. Arquitetura Geral
5. Estrutura do Mundo
6. Gameplay Architecture
7. Core Systems
8. Save System
9. Convenções
10. Ordem de Implementação

1. Objetivo

Este documento descreve toda a arquitetura técnica utilizada no desenvolvimento de ****The Labyrinth****.

Seu propósito é servir como referência durante a implementação, definindo como os sistemas do jogo serão organizados e como eles se comunicam entre si.

Este documento não descreve mecânicas de gameplay, narrativa, level design ou direção artística. Essas informações pertencem ao GDD e ao LDD.

Sempre que houver dúvidas sobre organização do código, estrutura do projeto ou fluxo de implementação, este documento deve ser utilizado como referência.

O principal objetivo é manter o projeto simples, consistente e de fácil manutenção durante todo o desenvolvimento.

2. Stack do Projeto

Engine

- Godot Engine 4.x

Linguagem

- GDScript

Controle de Versão

- Git

Modelagem 3D

- Blender

Pixel Art / Interface

- Aseprite

IA de Desenvolvimento

- Claude Code

Plataforma Inicial

- Windows

Plataforma Alvo

- PC (Windows)

O projeto será desenvolvido inicialmente para computadores desktop. A arquitetura deverá permanecer independente de plataforma sempre que possível, permitindo futuras adaptações para outras plataformas sem alterações estruturais significativas.

3. Estrutura de Pastas

A organização do projeto busca separar claramente recursos, código e documentação. Cada pasta possui uma responsabilidade específica, evitando que arquivos de naturezas diferentes fiquem misturados.

...

res://

- |
- |— assets/
- |— audio/
- |— docs/
- |— resources/
- |— scenes/

```
|— scripts/
|— shaders/
|— ui/
|— addons/
|— project.godot
...

```

assets/

Contém todos os recursos visuais utilizados pelo jogo.

```
...
assets/
|
|— models/
|— textures/
|— materials/
|— vegetation/
|— props/
|— architecture/
|— icons/
...

```

models/

Modelos 3D em formato `.glb`.

Exemplos:

- Árvores
- Bancos
- Muros
- Estátuas
- Gazebos
- Portões

textures/

Texturas utilizadas pelos materiais.

Exemplos:

- Pedra
- Madeira
- Vegetação
- Solo

- Metal

materials/

Materiais reutilizáveis da Godot.

vegetation/

Objetos relacionados à vegetação.

Exemplos:

- Sebes
- Flores
- Arbustos
- Árvores
- Grama

props/

Objetos decorativos.

Exemplos:

- Bancos
- Vasos
- Placas
- Lanternas
- Cercas

architecture/

Estruturas arquitetônicas.

Exemplos:

- Pontes
- Gazebos
- Fontes
- Torres

- Muros
- Portões

audio/

Todo o conteúdo sonoro do projeto.

...

audio/

|

|— ambience/

|— music/

|— sfx/

...

ambience/

Sons ambientes.

Exemplos:

- Vento
- Pássaros
- Folhas
- Água

music/

Trilhas musicais.

sfx/

Efeitos sonoros.

Exemplos:

- Passos
- Portas
- Alavancas
- Interações

docs/

Documentação oficial do projeto.

...

docs/

|

|— 01_GDD.md

|— 02_TDD.md

|— 03_LDD.md

|— 04_TASKS.md

...

resources/

Resources utilizados pela Godot.

...

resources/

|

|— world/

|— regions/

|— objects/

|— puzzles/

|— events/

...

Esses arquivos armazenam apenas dados utilizados pelo jogo.

Nenhum comportamento deverá ser implementado nesta pasta.

scenes/

Todas as cenas do projeto.

...

scenes/

|

|— player/

|— world/

|— objects/

|— puzzles/

|— ui/

```
└─ menus/
...

```

Cada sistema importante deverá possuir sua própria pasta.

scripts/

Todo o código-fonte do projeto.

...

```
scripts/
|
├─ core/
├─ player/
├─ world/
├─ interaction/
├─ puzzles/
├─ events/
├─ save/
├─ ui/
└─ debug/
...

```

core/

Sistemas globais.

Exemplos:

- GameManager
- SceneManager
- AudioManager
- SaveManager

player/

Todo o código relacionado ao jogador.

world/

Carregamento do mundo, regiões e objetos.

interaction/

Sistema de interação entre jogador e ambiente.

puzzles/

Lógica base dos puzzles.

events/

Sistema de eventos raros.

save/

Persistência dos dados do jogo.

ui/

Interface do usuário.

debug/

Ferramentas utilizadas apenas durante o desenvolvimento.

shaders/

Shaders personalizados.

ui/

Recursos visuais da interface.

Exemplos:

- Ícones
- Fontes
- Painéis
- Cursores

Organização Geral

As seguintes regras devem ser respeitadas durante todo o desenvolvimento:

- Cada sistema deve possuir sua própria pasta.
- Nenhum script deve ficar diretamente na raiz do projeto.
- Assets nunca devem ser misturados com código.
- Scenes nunca devem conter lógica que pertença aos scripts.
- Resources devem armazenar apenas dados.
- Toda documentação deve permanecer dentro da pasta `docs`.

4. Arquitetura Geral

A arquitetura de **The Labyrinth** foi projetada para ser simples, modular e de fácil manutenção.

Cada sistema do jogo possui uma responsabilidade bem definida e deve interagir com os demais através de interfaces claras, evitando dependências desnecessárias.

O objetivo não é criar uma arquitetura extremamente genérica, mas uma estrutura que facilite o desenvolvimento do jogo e permita futuras expansões.

Visão Geral

O jogo é dividido em cinco grandes sistemas.

...

Player

↓

Interaction

↓

World

↓

Objects

↓

Save

'''

Cada um possui responsabilidades específicas.

Player

Responsável exclusivamente pelo personagem controlado pelo jogador.

Inclui:

- movimentação;
- câmera;
- entrada de comandos;
- interação com o ambiente.

O Player nunca controla diretamente puzzles, portas ou eventos.

Sua única função é enviar ações para o restante do jogo.

Interaction

O sistema de interação funciona como intermediário entre o jogador e o mundo.

Fluxo:

'''

Jogador

↓

Raycast

↓

Objeto

↓

Interação

'''

Sempre que o jogador realizar uma interação, o sistema identifica o objeto à frente e solicita que ele execute sua própria lógica.

Isso evita que o Player precise conhecer detalhes dos objetos existentes no mapa.

World

O World representa todo o cenário carregado na partida.

É responsável por:

- carregar regiões;
- posicionar objetos;
- controlar o ambiente;
- fornecer informações espaciais ao restante do jogo.

O World não contém regras de gameplay.

Seu objetivo é apenas representar o ambiente.

Objects

Todo elemento presente no cenário é considerado um objeto.

Exemplos:

- portas;
- bancos;
- fontes;
- estátuas;
- árvores especiais;
- placas;
- alavancas;
- mecanismos de puzzles.

Cada objeto é responsável apenas por seu próprio comportamento.

Sempre que possível, objetos reutilizarão componentes comuns para reduzir duplicação de código.

Save

O sistema de Save é responsável por armazenar apenas as alterações realizadas durante a partida.

O mapa original nunca é salvo.

São armazenados apenas estados modificados.

Exemplos:

- portas abertas;
- puzzles resolvidos;
- eventos concluídos;
- posição do jogador.

Fluxo Geral do Jogo

O funcionamento básico do jogo segue o fluxo abaixo.

...

Input

↓

Player

↓

Interaction

↓

Objeto

↓

Resposta

↓

Save (quando necessário)

...

Esse fluxo deve permanecer o mais simples possível durante todo o desenvolvimento.

Princípios da Arquitetura

Responsabilidade Única

Cada script deve possuir apenas uma responsabilidade.

Exemplos:

PlayerController

→ controla apenas o jogador.

Door

→ controla apenas a porta.

SaveManager

→ controla apenas o sistema de save.

Baixo Acoplamento

Sempre que possível, um sistema não deve depender diretamente de outro.

Por exemplo, o Player não precisa conhecer todos os tipos de portas existentes.

Ele apenas solicita uma interação.

O objeto decide como responder.

Reutilização

Sempre que uma funcionalidade puder ser reutilizada por vários objetos, ela deve ser implementada apenas uma vez.

Exemplos:

- sistema de interação;
- sistema de destaque (highlight);
- sons de interação;

- animações de abertura.

Simplicidade

Sempre que existirem duas soluções para um problema, deve ser escolhida a mais simples que atenda às necessidades do projeto.

Arquiteturas complexas só deverão ser adotadas quando houver uma necessidade real.

Estrutura Inicial das Cenas

O projeto utilizará uma cena principal responsável por inicializar o jogo.

...

Main

|

|— World

|— Player

|— UI

|— Audio

|— Managers

...

World

Contém todo o cenário carregado.

Player

Instância do personagem.

UI

Interface do usuário.

Audio

Controla música e sons ambientes.

Managers

Objetos responsáveis pelos sistemas globais.

Exemplos:

- SaveManager
- GameManager
- SceneManager
- AudioManager

5. Estrutura do Mundo

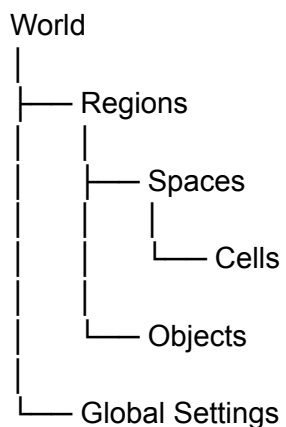
O mundo de ****The Labyrinth**** é organizado em uma hierarquia simples, permitindo que o mapa seja dividido em áreas menores e mais fáceis de manter.

Essa organização também facilita a implementação de novos conteúdos sem alterar a estrutura existente.

Hierarquia

Todo o mundo segue a estrutura abaixo.

...



...

Cada nível possui uma responsabilidade específica.

World

O World representa todo o mapa do jogo.

Existe apenas um World durante toda a partida.

Ele é responsável por:

- carregar as regiões;
- manter as configurações globais;
- armazenar informações gerais do ambiente.

O World não possui regras de gameplay.

Sua função é apenas organizar o cenário.

Regions

As Regions representam grandes áreas do labirinto.

Cada região possui identidade própria.

Exemplos:

- Entrance Garden
- Rose Garden
- Dead Forest
- Greenhouse
- Statue Garden
- Clock Tower

Cada Region poderá possuir:

- vegetação própria;
- iluminação própria;
- atmosfera própria;
- landmarks;
- puzzles;
- eventos raros.

As regiões são independentes entre si.

Spaces

Dentro de cada Region existem diversos Spaces.

Um Space representa um local que o jogador percebe como uma unidade.

Exemplos:

- corredor;
- praça;
- ponte;
- gazebo;
- lago;
- clareira;
- estufa.

O jogador nunca percebe Cells.

Ele percebe Spaces.

Por esse motivo, os Spaces são a principal unidade utilizada durante o Level Design.

Tipos de Space

Cada Space pertence a um dos seguintes tipos.

...

Linear

Decision

Compression

Expansion

Observation

Transition

Identity

Puzzle

Event

Secret

Memory

Silence

'''

Esses tipos ajudam a definir o ritmo da exploração e servem como referência durante a construção do mapa.

Cells

As Cells representam a menor unidade lógica do mapa.

Cada Cell contém informações sobre:

- posição;
- conexões;
- objetos presentes;
- tipo de terreno.

As Cells existem apenas para facilitar a construção do mundo.

O jogador nunca percebe sua existência.

Grid

Todo o mapa será construído utilizando um Grid.

Cada Cell ocupa uma posição fixa dentro desse Grid.

'''

```
+---+---+---+
|   |   |   |
+---+---+---+
|  X  |   |
+---+---+---+
|   |   |   |
+---+---+---+
'''
```

Essa organização facilita:

- construção do mapa;
- posicionamento de objetos;
- navegação;
- futuras ferramentas de edição.

Objetos

Todos os elementos existentes no cenário são tratados como objetos.

Exemplos:

- árvores;
- bancos;
- fontes;
- estátuas;
- portões;
- placas;
- alavancas;
- mecanismos de puzzles.

Os objetos pertencem sempre a uma Region e ocupam uma posição específica dentro do mapa.

Landmarks

Landmarks são elementos utilizados como referência visual para navegação.

Exemplos:

- Torre do Relógio;
- Fonte Central;
- Grande Estátua;
- Gazebo Principal;
- Lago Seco.

Todo Landmark deve ser facilmente reconhecível à distância.

Além de enriquecer o cenário, eles ajudam o jogador a criar um mapa mental do labirinto.

Carregamento do Mundo

Ao iniciar uma partida, o carregamento seguirá a sequência abaixo.

...

Inicialização

↓

World

↓

Regions

↓

Objects

↓

Player

↓

Interface

↓

Gameplay

'''

Essa ordem garante que o cenário esteja completamente carregado antes que o jogador possa interagir com ele.

Organização dos Objetos

Os objetos deverão ser agrupados por categoria dentro da cena de cada Region.

Exemplo:

'''

Region

|

|— Environment

|— Vegetation

|— Architecture

|— Props

|— Puzzles

|— Events

'''

Essa organização facilita tanto o desenvolvimento quanto a localização de elementos dentro da Godot.

6. Gameplay Architecture

Esta seção descreve os principais sistemas responsáveis pelo funcionamento do jogo.

Cada sistema possui uma responsabilidade específica e deve permanecer o mais independente possível dos demais.

O objetivo é manter o código organizado, reutilizável e fácil de manter durante todo o desenvolvimento.

Player

O Player representa o personagem controlado pelo jogador.

Ele é composto pelos seguintes módulos.

...

Player

|

|— Input

|— Movement

|— Camera

|— Interaction

...

Cada módulo possui uma responsabilidade única.

Input

Responsável por capturar todas as ações do jogador.

Exemplos:

- movimentação;
- olhar;
- interação;
- pausa.

O sistema de Input apenas identifica os comandos.

A execução dessas ações pertence aos sistemas responsáveis.

Movement

Responsável pela movimentação do jogador.

Inclui:

- deslocamento;
- velocidade;
- colisão;
- gravidade;
- corrida (caso implementada futuramente).

O sistema de movimento não deve conhecer objetos do cenário.

Camera

Responsável pela câmera em primeira pessoa.

Funções:

- rotação horizontal;
- rotação vertical;
- limite de inclinação;
- suavização (caso utilizada).

A câmera permanece independente da lógica do jogo.

Interaction

Toda interação entre jogador e cenário ocorre através deste sistema.

Fluxo:

...

Jogador

↓

Raycast

↓

Objeto



Interação

Sempre que possível, apenas um objeto poderá ser interagido por vez.

O objeto decide como responder à interação.

Objetos Interativos

Todo objeto interativo deverá possuir uma interface de interação comum.

Exemplos:

- portas;
- placas;
- alavancas;
- mecanismos;
- elementos de puzzles.

Isso permite que o sistema de interação permaneça simples e reutilizável.

Puzzles

Os puzzles são sistemas independentes.

Cada puzzle será responsável apenas pela sua própria lógica.

O jogador apenas envia interações.

Os puzzles decidem:

- quando iniciar;
- quando concluir;
- quando falhar.

Essa separação facilita a criação de novos puzzles sem alterar sistemas existentes.

Eventos Raros

Eventos raros representam acontecimentos ocasionais durante a exploração.

Exemplos:

- sons inesperados;
- mudanças sutis no ambiente;
- aparições temporárias;
- pequenas alterações visuais.

Todos os eventos deverão ser independentes do progresso principal do jogo.

Seu objetivo é aumentar a imprevisibilidade da exploração sem bloquear o avanço do jogador.

Interface

A interface do jogo será mínima.

Elementos previstos para o MVP:

- menu principal;
- menu de pausa;
- configurações;
- tela inicial.

Durante a exploração, a HUD deverá permanecer limpa, exibindo apenas informações realmente necessárias.

Fluxo Geral do Gameplay

...

Input

↓

Player

↓

Interaction

↓

Objeto ou Puzzle

↓

Resposta

↓

Atualização do Save (quando necessário)

...

Todo novo sistema implementado deverá se integrar a esse fluxo, evitando criar caminhos paralelos para execução de lógica.

7. Core Systems

Os Core Systems são responsáveis por funcionalidades globais do jogo.

Eles não implementam mecânicas de gameplay, mas fornecem suporte para o restante do projeto.

GameManager

O GameManager controla o estado geral do jogo.

Responsabilidades:

- iniciar uma nova partida;
- carregar uma partida salva;
- retornar ao menu principal;
- controlar o estado atual do jogo.

O GameManager não deve conter regras de gameplay.

SaveManager

Responsável por salvar e carregar o progresso do jogador.

Funções:

- criar arquivos de save;
- carregar saves existentes;

- atualizar informações persistentes;
- gerenciar múltiplos slots (caso implementado futuramente).

Todo o sistema de persistência deverá passar por este gerenciador.

AudioManager

Controla todos os sons do jogo.

Responsabilidades:

- música ambiente;
- efeitos sonoros;
- volume geral;
- volume de categorias;
- transições entre músicas.

Nenhum objeto deverá reproduzir sons diretamente sem utilizar este sistema.

SceneManager

Responsável pelas transições entre cenas.

Exemplos:

- Menu Principal;
- Gameplay;
- Configurações;
- Créditos.

Seu objetivo é centralizar toda a navegação entre telas.

Configurações

As configurações do jogo deverão ser centralizadas em um único sistema.

Exemplos:

- resolução;
- modo de tela;
- volume;
- sensibilidade do mouse.

Sempre que possível, essas configurações deverão ser persistidas entre sessões.

Comunicação entre Sistemas

Os sistemas globais devem possuir o menor acoplamento possível.

Sempre que possível, a comunicação deverá ocorrer através de chamadas simples e bem definidas, evitando dependências desnecessárias entre módulos.

O objetivo é manter o projeto fácil de modificar e expandir ao longo do desenvolvimento.

9. Convenções

As convenções definidas neste documento deverão ser seguidas durante todo o desenvolvimento do projeto.

O objetivo é manter o código consistente, organizado e fácil de compreender.

Nomenclatura

Classes

Classes deverão utilizar **PascalCase**.

Exemplos:

...

PlayerController

GameManager

SaveManager

Door

ClockTower

...

Variáveis

Variáveis deverão utilizar **camelCase**.

Exemplos:

...

```
moveSpeed
currentRegion
interactionDistance
playerPosition
'''
```

Constantes

Constantes deverão utilizar letras maiúsculas separadas por "_".

Exemplos:

```
'''
MAX_SPEED
DEFAULT_FOV
PLAYER_HEIGHT
'''
```

Funções

Funções deverão utilizar verbos que descrevam claramente sua ação.

Exemplos:

```
'''
move()

interact()

saveGame()

loadGame()

openDoor()

startPuzzle()
'''
```

Organização das Cenas

Cada cena deverá representar apenas um elemento do jogo.

Exemplos:

- Player
- Door
- Statue
- Fountain
- Puzzle
- Region

Evitar cenas excessivamente grandes ou que concentrem responsabilidades diferentes.

Organização dos Scripts

Cada script deverá possuir apenas uma responsabilidade.

Evitar arquivos muito longos.

Sempre que um script começar a concentrar diversas funções diferentes, avaliar sua divisão em novos scripts.

Organização dos Assets

Os assets deverão permanecer agrupados por categoria.

Exemplo:

```

models/

textures/

materials/

vegetation/

props/

architecture/

```

Evitar misturar arquivos de naturezas diferentes na mesma pasta.

Comentários

Comentários devem explicar decisões ou comportamentos que não sejam imediatamente evidentes.

Evitar comentar código óbvio.

Dar preferência a nomes claros para variáveis, funções e classes em vez de depender de comentários.

Princípios Gerais

Durante o desenvolvimento, buscar sempre:

- simplicidade;
- legibilidade;
- reutilização;
- baixo acoplamento;
- organização.

Sempre que existirem duas soluções equivalentes, optar pela mais simples.

10. Ordem de Implementação

O desenvolvimento seguirá uma sequência incremental.

Cada etapa deverá estar funcional antes do início da próxima.

O objetivo é construir uma versão jogável do jogo o mais cedo possível.

Fase 1 — Fundação

- Configuração do projeto
- Estrutura de pastas
- Controle de versão
- Cena principal

Fase 2 — Player

- Input
- Movimento
- Câmera

- Colisão

Ao final desta etapa, o jogador deverá conseguir explorar um cenário vazio.

Fase 3 — Interação

- Raycast
- Sistema de interação
- Objetos interativos básicos

Ao final desta etapa, será possível interagir com elementos simples do cenário.

Fase 4 — Graybox

- Construção inicial do labirinto
- Regiões
- Caminhos
- Landmarks
- Navegação

Toda a estrutura do mapa será criada utilizando geometria simples, sem preocupação com arte final.

Fase 5 — Sistemas Base

- Save
- Menu Principal
- Menu de Pausa
- Configurações

Fase 6 — Gameplay

- Framework de puzzles
- Eventos raros
- Objetos especiais

Fase 7 — Arte

- Modelos Low Poly
- Materiais
- Vegetação
- Iluminação
- Sons ambientes

Fase 8 — Polimento

- Correção de bugs
- Ajustes de gameplay
- Otimizações
- Balanceamento
- Melhorias visuais

MVP

O projeto será considerado em estado de MVP quando atender aos seguintes critérios:

- O jogador consegue iniciar uma nova partida.
- O labirinto completo pode ser explorado.
- A movimentação está funcional.
- O sistema de interação está implementado.
- Os principais puzzles estão jogáveis.
- O sistema de save funciona corretamente.
- Existe início e fim de jogo.

A partir desse ponto, novas funcionalidades deverão ser adicionadas apenas se contribuírem para melhorar a experiência principal de exploração.

Considerações Finais

Este documento define a arquitetura técnica de **The Labyrinth**.

Seu objetivo é servir como referência durante todo o desenvolvimento do projeto, garantindo consistência na organização do código e na implementação dos sistemas.

Sempre que uma decisão técnica importante modificar a arquitetura descrita neste documento, ele deverá ser atualizado para refletir a nova realidade do projeto.